
Software Requirements Specification

for

HostelMate

(Hostel Management System)

Version 1.0

Prepared by

Harmi Kotak	23CEUOG074	23ceuog074@ddu.ac.in
Vidhi Ladani	24CEUOG078	24ceuog078@ddu.ac.in

Instructor: Jatayu Baxi

Course: Computer Engineering

Lab Section: A1

Teaching Assistant: *Jatayu Baxi*

Date: 2nd April, 2025

CONTENTS	II
REVISIONS	III
1 INTRODUCTION	1
1.1 DOCUMENT PURPOSE.....	1
1.2 PRODUCT SCOPE.....	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW	1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	2
1.5 DOCUMENT CONVENTIONS	2
1.6 REFERENCES AND ACKNOWLEDGMENTS.....	2
2 OVERALL DESCRIPTION	3
2.1 PRODUCT OVERVIEW	3
2.2 PRODUCT FUNCTIONALITY	3
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS.....	4
2.4 ASSUMPTIONS AND DEPENDENCIES	4
3 SPECIFIC REQUIREMENTS	5
3.1 EXTERNAL INTERFACE REQUIREMENTS	5
3.2 FUNCTIONAL REQUIREMENTS.....	6
3.3 USE CASE MODEL	11
4 OTHER NON-FUNCTIONAL REQUIREMENTS	15
4.1 PERFORMANCE REQUIREMENTS	15
4.2 SAFETY AND SECURITY REQUIREMENTS.....	15
4.3 SOFTWARE QUALITY ATTRIBUTES	15
5 OTHER REQUIREMENTS	
APPENDIX A – DATA DICTIONARY	16
APPENDIX B - GROUP LOG	16

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Vidhi Ladani & Harmi Kotak	First Version	1 st April, 2025

•

1 Introduction

1.1 Document Purpose

This Software Requirements Specification (SRS) document provides a detailed description of the functional and nonfunctional requirements for the HostelMate application. It is intended to be used by the manager of the hostel and the hostelites (stakeholders) along with the software development team, its testers to ensure that the system meets all specified requirements.

1.2 Product Scope

The **HostelMate** allows hostel managers to manage the hostelites data and ensures smooth functioning of the hostel and also makes it easier for hostelites to avail the gate pass generation feature and provide detailed feedback of the food. The system helps the manager to analyse the feedback in detail, along with processing the requests of changing rooms and generating gatepasses. Therefore the system has a great potential to digitalise the management of hostels and give a hassle free experience to its users.

1.3 Intended Audience and Document Overview

- The intended audience of this document comprises the software developers involved in this project, the users of the product along with the testing team of the final product that is to be delivered.
- The document further describes the functional and non-functional requirements of the software. The functional requirements are described user wise. The non-functional requirements describe the hardware, software interfaces, implementation techniques along with the safety and security measures that were taken care of.
- The readers should follow the specific sequence in which the document is formed in order to gain a better and clear understanding of the content written hereby.

1.4 Definitions, Acronyms and Abbreviations

- User: Any person who interacts with the system.
- Admin: An authorized user who manages the HostelMate
- SRS: Software Requirements Specification.
- API: Application Programming Interface.

1.5 Document Conventions

- This document follows the IEEE formatting requirements.
- Bold and underlined items denotes the section titles of the document
- Bold and italics items denoted the subsections of the titles
- 1, 2, 3.. numbering is used for denoting the main titles
- 1.1, 2.3, 4.4... numbering is used for their subsections for easy identification

1.6 References and Acknowledgments

This document references the following resources:

- [IEEE SRS Template](#)
- Relevant user interface design guidelines
- Documentation for integration with e-commerce platforms (e.g., Amazon, Flipkart, Shopify)
- Research papers on machine learning algorithms for recommendation systems

2 Overall Description

2.1 Product Overview

- This product might be there, already existing somewhere but it will not be exactly the same as this product because we will be including the feature of gate pass generation.
- The feature of gate pass generation will make this product different from all the other software applications of HostelMate which are there in existence right now

2.2 Product Functionality

- Sign-in: The user will be allowed to sign in with their username and password
- Sign-up: For the first time, the user has to sign-up using all the basic details which includes parents details, mobile number, residential details, educational details and other required information including room number and bed number.
- Profile details: The profile details can be viewed.
- Hostelites data: The data of all the registered hostelites can be visible
- Payment status: Pending and paid status of the payment is visible
- Vacancy list: The number of vacancies can be seen through this function
- Complain box: The drafted complains are visible
- Feedback Analysis: Meal to meal the feedback provided by the users can be analyzed.
- Gate pass generation: Gate pass can be generated by giving the details of "TO" to "FROM"

2.3 Design and Implementation Constraints

- The application of the user will be blocked for 2 hours if the user enters the incorrect password during sign in for more than 3 times

2.4 Assumptions and Dependencies

- The application will work smoothly without the internal disturbances and the user will enter the correct details only.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The user interface for the HostelMate is designed to be intuitive and user-friendly. The primary interface will be a web-based platform accessible via desktop or mobile browsers. Users will interact with the system using:

- **User friendly homepage:** An effective user friendly home page for users to easily navigate through various functions.
- **Interactive Buttons:** Interactive buttons to send request, complaints, etc.

3.1.2 Hardware Interfaces

The hardware interfaces required for the HostelMate include the following components:

- **Web Servers:** The system interacts with web servers to handle user requests, serve content, and manage user sessions.
- **Database Servers:** Interfaces with a database management system to store and retrieve hostelite data, manage history, and past records.

3.1.3 Software Interfaces

The HostelMate connects with the following software components:

Cloud components: While the primary focus is the web-based platform, the system will integrate with a cloud resource for effective storage of data.

Analytics Tools: Interfaces with analytics software to track feedback analysis and present it using concise graph based patterns.

3.2 Functional Requirements

1. Login

R.1.1 Sign-in

- **Description:** The sign-in function asks the user whether they are an admin or a hostelite. If the user selects "admin," an extra option for a security key appears. The user must enter a username and password to log in.
- **State:** Either admin or hostelite is signed in.
- **Input:** The username, password, security key (if admin).
- **Output:** The dashboard page opens.
- **Processing:** Checks if the username and password are correct. If incorrect, a message prompts the user to re-enter the correct credentials.

R.1.2 Sign-up

R.1.2.1 Personal Information

- **Description:** This function requires the user to enter all personal details, educational details, residential details, and guardian details.
- **State:** Hostelite is signed up.
- **Input:** User's personal details, username, and password.
- **Output:** The dashboard page opens.
- **Processing:** The profile details are recorded in the database.

2. Dashboard

R.2.1 Hostelites Data

R.2.1.1 View Hostelites Details

- **Description:** The admin can view details of all hostelites.
- **State:** Admin is signed in and clicked on "Hostelites Data."
- **Input:** Click on a hostelite's name (which contains a hyperlink).

- **Output:** The corresponding hostelite's details are displayed.
- **Processing:** Redirects to a page containing the hostelite's details.

R.2.1.2 My Profile Details

- **Description:** Displays the details of the logged-in hostelite.
- **State:** Hostelite is signed in and clicked on "My Profile."
- **Input:** Click on the profile data from the dashboard.
- **Output:** Displays the personal details of the hostelite.
- **Processing:** Fetches the hostelite's details from the database.

R.2.2 Vacancy List

- **Description:** Displays the list of vacant rooms.
- **State:** Either admin or hostelite is signed in and clicked on "Vacancy List."
- **Input:** Click on "Vacancy List" from the dashboard.
- **Output:** Displays the list of available rooms with room and bed numbers.
- **Processing:** Fetches all vacant rooms from the database.

R.2.3 Request Management

R.2.3.1 View Requests

- **Description:** Displays all requests submitted by hostelites.
- **State:** Admin is signed in and clicked on "Requests."
- **Input:** Click on "Requests" from the dashboard.
- **Output:** Displays pending requests.
- **Processing:** Fetches pending requests and displays them with details.

R.2.3.2 Submit Requests

- **Description:** Allows hostelites to submit requests for maintenance or complaints.
- **State:** Hostelite is signed in and clicked on "Requests."
- **Input:** Click on "Requests" and enter details.
- **Output:** A message is displayed confirming the request submission.
- **Processing:** Stores request details in the database.

R.2.4 Feedback System

R.2.4.1 Daily Feedback Submission

- **Description:** Displays a feedback form for breakfast, lunch, and dinner.
- **State:** Hostelite is signed in and clicked on "Feedback."
- **Input:** Select a meal type and provide a rating.
- **Output:** The feedback form is submitted successfully.
- **Processing:** Stores the submitted feedback in the database.

R.2.4.2 Feedback Analysis Graphs

- **Description:** Admin can view a visual analysis of feedback using bar graphs and AI-based sentiment analysis.
- **State:** Admin is signed in and clicked on "Feedback Analysis."
- **Input:** Click on "Feedback Analysis."
- **Output:** Displays bar graphs and an overall sentiment analysis graph.
- **Processing:** Fetches feedback data, analyzes it using TextBlob and NLTK, and generates visual graphs.

R.2.5 Gatepass Management

R.2.5.1 View Active Gatepasses

- **Description:** Displays currently active gatepasses.
- **State:** Admin is signed in and clicked on "Gatepass."
- **Input:** Click on "View Active Gatepasses."
- **Output:** Displays a list of active gatepasses. If none are active, a message is shown.
- **Processing:** Fetches all active gatepasses from the database.

R.2.5.2 Generate Gatepass

- **Description:** Allows hostelites to generate a gatepass for leaving the hostel.
- **State:** Hostelite is signed in and clicked on "Gatepass."
- **Input:** Click on "Generate Gatepass" and enter details such as reason, departure date, and return date.
- **Output:** The gatepass is successfully generated.
- **Processing:** If details are valid, the gatepass is stored in the database; otherwise, an error message is shown.

R.2.6 User Management (Admin Only)

R.2.6.1 View Users

- **Description:** Allows the admin to view details of all registered users.
- **State:** Admin is signed in and clicked on "User Management."
- **Input:** Click on a user's name.
- **Output:** Displays user details.
- **Processing:** Fetches user details from the database.

R.2.6.2 Update User Information

- **Description:** Admin can update hostelite details if necessary.
- **State:** Admin is signed in and clicked on "Edit User."
- **Input:** Enter new details and submit.
- **Output:** Displays a success message upon update.
- **Processing:** Updates the user's details in the database.

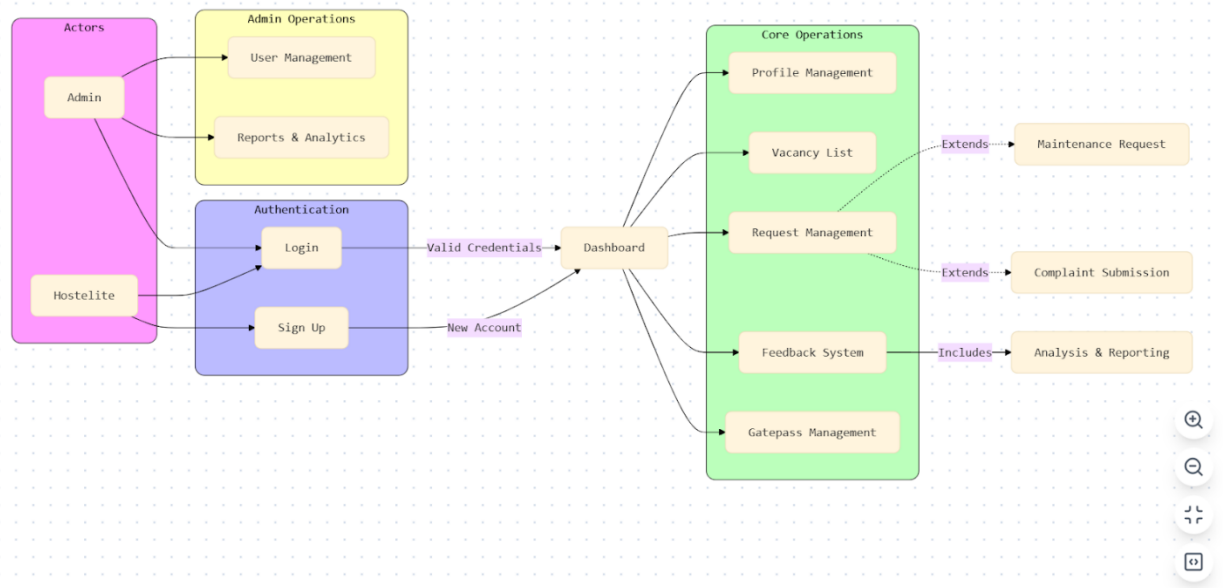
R.2.6.3 Delete User

- **Description:** Allows the admin to remove a user from the system.
- **State:** Admin is signed in and clicked on "Delete User."
- **Input:** Click "Delete" on a specific user.
- **Output:** Displays a confirmation message before deletion.
- **Processing:** Deletes the user from the database upon confirmation.

R.2.7 Logout**R.2.7.1 Logout Function**

- **Description:** Allows both hostelites and admins to log out of the system.
- **State:** User is signed in and clicked on "Logout."
- **Input:** Click "Logout."
- **Output:** Redirects to the login page.
- **Processing:** Ends the user session and redirects to the login page.

3.3 Use Case Model



Use Case 1: Login

- **Summary:** Enables users to log in to the system.
- **Actors:** Admin, Hostelites
- **Preconditions:** The user must be registered in the system.
- **Description:**
 - The user selects their role (Admin or Hostelite).
 - If the user is an Admin, an additional security key is required.
 - The user enters the username and password.
 - The system validates the credentials.
- **Exceptions:**
 - Incorrect username or password is entered.
 - The user tries to log in without signing up.
- **Postconditions:** The user is redirected to the respective dashboard based on their role.

Use Case 2: Sign-up

- **Summary:** Allows hostelites to register in the system.
 - **Actors:** Hostelites
 - **Preconditions:** The user should not be already registered.
 - **Description:**
 - The hostelite provides personal, educational, and residential details.
 - The hostelite creates a username and password.
 - The details are stored in the database.
 - **Exceptions:**
 - The hostelite is already registered.
 - An admin mistakenly tries to sign up.
 - **Postconditions:** The user is prompted to log in with the newly created credentials.
-

Use Case 3: View and Draft Complaints

- **Summary:** Enables hostelites to submit complaints and allows the admin to view them.
 - **Actors:** Admin, Hostelites
 - **Preconditions:** The user must be logged in and navigate to the complaint section.
 - **Description:**
 - Hostelites enter complaint details along with their identity.
 - The system stores the complaint in the database.
 - The admin can view all complaints submitted by hostelites.
 - **Exceptions:**
 - Incomplete complaint details.
 - Incorrect identity details entered.
 - **Postconditions:**
 - Hostelites receive a confirmation message that their complaint is submitted.
 - Admins can review complaints on the admin dashboard.
-

Use Case 4: View and Draft Requests

- **Summary:** Allows hostelites to send requests and enables the admin to review them.
- **Actors:** Admin, Hostelites
- **Preconditions:** The user must be logged in and navigate to the request section.
- **Description:**

- **Hostelites enter their request details along with their identity.**
- **The system records the request in the database.**
- **The admin can view all pending requests.**
- **Exceptions:**
 - **Incomplete or invalid request details.**
 - **Incorrect identity details entered.**
- **Postconditions:**
 - **Hostelites receive a confirmation message that their request is submitted.**
 - **Admins can review and process requests.**

Use Case 5: View Vacancies

- **Summary: Allows hostelites and admins to view available vacant rooms and beds.**
- **Actors: Admin, Hostelites**
- **Preconditions: The user must be logged in and navigate to the vacancies section.**
- **Description:**
 - **The user views the list of vacant rooms and beds.**
 - **The system fetches and displays available rooms and beds.**
- **Exceptions:**
 - **No vacancies available.**
- **Postconditions:**
 - **Users can see the list of available rooms and beds.**

Use Case 6: View or Generate Gate Pass

- **Summary: Allows hostelites to generate a gate pass and enables admins to view active gate passes.**
- **Actors: Admin, Hostelites**
- **Preconditions: The user must be logged in and navigate to the gate pass section.**
- **Description:**
 - **Hostelites provide details for generating a gate pass (reason, dates, guardian confirmation, etc.).**
 - **The system verifies the details and generates the gate pass.**
 - **Admins can view all active gate passes.**
- **Exceptions:**
 - **Incorrect or incomplete details.**
 - **Invalid parent contact details.**
 - **User enters incorrect travel dates.**

- **Postconditions:**
 - The hostelite receives confirmation that the gate pass is generated.
 - Admins can monitor active gate passes.

Use Case 7: Provide and Analyze Feedback

- **Summary:** Enables hostelites to provide meal feedback and allows admins to analyze it.
- **Actors:** Admin, Hostelites
- **Preconditions:** The user must be logged in and navigate to the feedback section.
- **Description:**
 - Hostelites provide ratings for breakfast, lunch, and dinner.
 - The system stores the feedback.
 - Admins can analyze the feedback using AI-based sentiment analysis and view graphical representations.
- **Exceptions:**
 - Insufficient feedback provided.
- **Postconditions:**
 - Hostelites receive a confirmation that their feedback is submitted.
 - Admins can access feedback reports and analysis.

Use Case 8: QR Scanner for Entry/Exit

- **Summary:** Enables hostelites to scan QR codes when entering or exiting the hostel.
- **Actors:** Hostelites
- **Preconditions:** The user must be logged in and navigate to the QR scanner.
- **Description:**
 - The hostelite scans the QR code while exiting the hostel.
 - The hostelite scans the QR code again upon returning.
- **Exceptions:**
 - Hostelite forgets to scan when exiting.
 - System failure prevents scanning.
- **Postconditions:**
 - Hostelite receives confirmation of a successful scan.

4 Other Non-functional Requirements

4.1 Performance Requirements

- *All pages of the website should take less than one second to generate*
- *Minimization of required bandwidth using industry-standard compression*
- *Database queries should be optimized to ensure a **fast response time** for retrieving and storing data.*
- *The system should efficiently handle **simultaneous user requests** without significant delays.*

4.2 Safety and Security Requirements

- ***Data Encryption:** All user data is encrypted and is safe*
- ***Secure Authentication:** Use secure login password to protect user accounts.*

4.3 Software Quality Attributes

- **Performance:** The system should respond to user interactions within 2 seconds under normal load.
- **Reliability:** Ensure 99.9% uptime with mechanisms for fault detection and recovery.
- **Scalability:** Support an increasing number of users and data without performance degradation.
- **Usability:** Provide a user-friendly interface with intuitive navigation and multi-language support.
- **Maintainability:** Use modular design and adhere to coding standards to simplify updates and bug fixes.

Appendix A – Data Dictionary

<Data dictionary is used to track all the different variables, states and functional requirements that you described in your document. Make sure to include the complete list of all constants, state variables (and their possible states), inputs and outputs in a table. In the table, include the description of these items as well as all related operations and requirements.>

Appendix B - Group Log

<Please include here all the minutes from your group meetings, your group activities, and any other relevant information that will assist in determining the effort put forth to produce this document>